

AGENT ON A LEASH

Hackathon Reference Guide

Wallet Control Layer for AI Shopping Agents

Prepared: September 18, 2026
Viseca × Hackathon Challenge

TABLE OF CONTENTS

1. Project Overview
2. Challenge Objective
3. Key Requirements
4. System Architecture
5. Demonstration Criteria
6. Technical Preferences
7. Support & Resources
8. Solution Checklist

1. PROJECT OVERVIEW

About Viseca

Viseca is a leading Swiss fintech specializing in payment cards and cashless payment services. Founded in 1999 and wholly owned by Swiss retail and cantonal banks, Viseca combines decades of payment expertise with a focus on making payments simple, secure, and convenient.

The Challenge Context

Imagine an AI shopping assistant that can buy things with your credit card. Such agents are already a reality, enabled by new capabilities from Visa and Mastercard.

Build the trust and control layer that decides whether an AI agent may spend a customer's money.

Safety Concerns to Address

- Spending Limits
- Merchant Vetting
- Intent Verification
- Security

2. CHALLENGE OBJECTIVE

Build a prototype wallet control layer that decides whether an AI shopping agent may spend a customer's money.

Key Workflow

- 1. Customer sends shopping request
- 2. Define wallet policy
- 3. Forward transaction to Wallet Control
- 4. Evaluate against policy
- 5. Return decision: approve/decline/ask

■ YOU BUILD	■ YOU DON'T BUILD
Wallet control layer	Shopping agent
Policy interpretation	Payment processing
Transaction evaluation	Merchant databases

3. KEY REQUIREMENTS

Must-Have Capabilities

- Policy Translation
- Policy Management
- Transaction Evaluation
- Decision Engine

Transparency & Auditability

Judges must understand: what permitted, evidence considered, why it acted, customer control retained

4. SYSTEM ARCHITECTURE

Technical Preferences

Decouple wallet-control UI from decision engine for independent deployment

- If using language models: prefer smaller, lower-latency models
- System must function if optional models fail

5. DEMONSTRATION CRITERIA

Scenario 1: Ordinary transaction (straightforward, approves)

Scenario 2: Suspicious transaction (useful intervention)

Scenario 3: Human approval/rejection path

6. TECHNICAL PREFERENCES

Combine rules, behavioral signals, ML, language models, and good UX

7. SUPPORT & RESOURCES

- Synthetic data pack available on Github
- Live API with transactions during hackathon
- Architecture diagram in Github Repo

8. SOLUTION CHECKLIST

Core Components:

- Policy interpretation engine
- Transaction evaluation logic
- Decision engine
- Human approval path
- Policy management

Testing & Validation:

- Ordinary transaction
- Suspicious transaction
- Human escalation
- Synthetic data
- Live API

Demonstration Ready:

- Transaction flow demo
- Intervention demo
- Human path demo
- Explainable decisions
- Architecture doc